Allo Health

TECHNICAL TAKE-HOME ASSIGNMENT SUBMISSION DOCUMENT

Name: Chethan N

SRN: PES2UG22CS148

■ Required Assignment Deliverables

Deliverable Name	Access Link
1. Public GitHub Repository: Full version-controlled source code and commit logs.	**https://github.com/chethan264/allo_health**
2. Live Production URL: Fully functional web storefront with hosted database.	**https://allo-health-11uk.vercel.app**
3. Cloud Database Engine: Hosted Serverless PostgreSQL database.	Neon PostgreSQL Cloud Instance (neondb)

■ Core Architectural Solutions

This platform successfully addresses the high-concurrency reservation race conditions using the following technical implementations:

- **Pessimistic Row-Level Locking (SELECT ... FOR UPDATE):** To prevent double-selling of low-stock items under flash-sale traffic, we acquire transactional database row-locks on the specific product-warehouse stock row. Concurrent checkout threads are safely blocked and queued at the database level rather than creating optimistic write-skew failures.
- **Resilient DB Connection Retry Engine (withDbRetry):** Serverless databases like Neon PostgreSQL go to sleep when idle. To shield shoppers from cold-start timeout spikes, we built an automatic query retry wrapper that intercepts connection failures, waits 1.5s, and automatically retries queries up to 3 times before returning an error.
- **Two-Pronged Expiry & Reclaim:** Open holds automatically expire in 10 minutes. Stock is reclaimed using (1) *Lazy Cleanups on Reads/Writes* for instant consistency on catalog views and (2) a background Vercel cron worker (`/api/cron/cleanup`).
- **API Idempotency Layer:** Safety checks using an `Idempotency-Key` header on checkout creations and confirmations prevent duplicate holds or charging in the event of client retries.

Allo Health

TECHNICAL SUBMISSION - CONCURRENCY VERIFICATION LOGS

■ Verified Concurrency Stress Test Results

We built a dedicated concurrent assertion suite in `scratch/stress_test.ts` to simulate intense flash-sale traffic. The test resets a Mumbai Logistics Hub stock level to exactly **1 unit**, fires **10 concurrent requests** at the exact same instant, and evaluates the outcomes:

```
=== STARTING CONCURRENCY STRESS TEST ===
Resetting "allo-limited-pack" stock level in Mumbai Hub to exactly 1 available unit...
Live stock in DB: Total=1, Reserved=0

Preparing 10 simultaneous reservation requests for the last 1 available unit...
Firing all 10 reservation requests at the same instant...
All concurrent requests completed. Evaluating responses...
Request #1: CONFLICT (409 Conflict) - Sold Out. Message: Insufficient stock...
Request #2: CONFLICT (409 Conflict) - Sold Out. Message: Insufficient stock...
Request #3: CONFLICT (409 Conflict) - Sold Out. Message: Insufficient stock...
Request #4: CONFLICT (409 Conflict) - Sold Out. Message: Insufficient stock...
Request #5: CONFLICT (409 Conflict) - Sold Out. Message: Insufficient stock...
Request #6: CONFLICT (409 Conflict) - Sold Out. Message: Insufficient stock...
Request #7: CONFLICT (409 Conflict) - Sold Out. Message: Insufficient stock...
Request #8: SUCCESS (201 Created). Reservation ID: 9571f165-72c1-42a8-954a-a5699f2b9c60
Request #9: CONFLICT (409 Conflict) - Sold Out. Message: Insufficient stock...
Request #10: CONFLICT (409 Conflict) - Sold Out. Message: Insufficient stock...

=== STRESS TEST RESULTS ANALYSIS ===
201 Created (Successes): 1 (Expected: 1) | 409 Conflict (Conflicts): 9 (Expected: 9)
Checking final stock state in database... Final stock in DB: Total=1, Reserved=1
Active pending reservations: 1

■ SUCCESS: High-concurrency pessimistic locking is fully robust! No race conditions,
no double-selling, and no negative inventory leaks occurred in the database.
```

■ Final Status Summary

The platform is fully complete and compiles perfectly. All deliverables strictly satisfy all take-home assignment criteria, complete with hosted Neon Cloud DB and live production Vercel deployment.